**Practical Activity: Student Marks Management System**

A lecturer needs a Java GUI application to capture, process, save, and retrieve student test marks. Three students each write two tests. The lecturer wants to enter the marks into the application and automatically calculate the total marks, average marks, number of passing marks, and number of failing marks.

The application must use a two-dimensional array, an interface, a class implementing the interface, Java Swing GUI components, file handling, and unit testing.

**Activity 1 – Plan your application**

**Task 1.1: Understand the data**

The application must initially be tested using the following data:

| Student | Test 1 | Test 2 |
|---|---|---|
| Student 1 | 70 | 60 |
| Student 2 | 40 | 80 |
| Student 3 | 30 | 90 |

The passing mark is 50%. A mark of 50 is considered a pass.

**How to approach this task**

Before writing any code, answer these questions:

1. How many rows will your array contain?
2. How many columns will your array contain?
3. What does each row represent?
4. What does each column represent?
5. What data type should store the marks?

**Why are you doing this?**

Planning your array structure first helps you understand how data will be stored and accessed.

**Activity 2 – Design the GUI**

**Task 2.1: Create the project**

Using NetBeans IDE:

1. Create a new Java Application project called StudentMarksApp.
2. Create a package called studentmarksapp.
3. Create a JFrame Form named StudentMarksGUI.
4. Open the Design view.

**Task 2.2: Add the GUI components**

Your application must contain:

| Component | Quantity | Purpose |
|---|---|---|
| JTextField | 6 | Capture test marks |
| JLabel | As needed | Identify fields and show students processed |
| JButton | 4 | Calculate, Save, Load, Clear |
| JTextArea | 1 | Display calculated results |
| JScrollPane | 1 | Contain the results area |

Use the NetBeans Palette to drag components onto your JFrame.

Give your components meaningful variable names. For example, a text field for Student 1's first test could be named txtStudent1Test1.

**Why?**

Meaningful variable names make it easier to identify components when writing event-handling code.

Checkpoint: Run your JFrame and confirm that all components appear correctly before continuing.

**Activity 3 – Create the interface**

**Task 3.1: Create IStudentMarks**

Create a Java interface named IStudentMarks.

Your interface must declare the following methods:

| Method | Return type | Responsibility |
|---|---|---|
| GetMarks() | int[][] | Return the marks array |
| GetTotalMarks() | int | Return the sum of all marks |
| GetAverageMarks() | double | Return the average |
| GetPassCount() | int | Return the number of passing marks |
| GetFailCount() | int | Return the number of failing marks |
| GetStudentsProcessed() | int | Return the number of students |

**How to approach this task**

Recall the basic method declaration structure:

returnType methodName();

Use this structure to declare each required method.

**Why?**

An interface establishes a contract that the implementing class must follow. It ensures that all required operations are provided.

**Activity 4 – Create the StudentMarks class**

**Task 4.1: Create the class**

Create a Java class named StudentMarks.

The class must implement the IStudentMarks interface.

**How to approach this task**

Use the Java keyword that connects a class to an interface.

Declare a private 2D integer array to store the marks.

Create a constructor that receives a 2D array and assigns it to the class field.

**Why?**

The GUI will capture the marks and pass them to the class. The class is responsible for processing the data, keeping the calculation logic separate from the interface design.

**Task 4.2: Calculate total marks**

Write the GetTotalMarks() method.

**Hints**

- Declare a variable to store the running total.
- Use an outer loop to process students.
- Use an inner loop to process each student's tests.
- Add each mark to the running total.
- Return the total after both loops finish.

**Why?**

Nested loops allow you to access every value in a two-dimensional array.

**Task 4.3: Calculate average marks**

Write the GetAverageMarks() method.

**Hints**

- Reuse the total calculation method.
- Determine how many test marks were processed.
- Ensure that the division produces a decimal result.
- Consider what should happen if the array is empty.

**Why?**

Reusing methods reduces code duplication and makes the application easier to maintain.

**Task 4.4: Count passing marks**

Write GetPassCount().

**Hints**

- Start a counter at zero.
- Visit every mark using nested loops.
- Check whether each mark is at least 50.
- Increase the counter when the condition is true.

**Why?**

This activity develops your understanding of conditional statements inside loops.

**Task 4.5: Count failing marks**

Write GetFailCount().

Use the same approach as the pass-count method, but change the condition to identify marks below 50.

**Task 4.6: Count students processed**

Write GetStudentsProcessed().

Hint: Each row represents one student. Consider which array property gives you the number of rows.

**Activity 5 – Connect the GUI to your class**

**Task 5.1: Capture the marks**

Create a method called captureMarks() in your JFrame class.

This method must read the six text fields and store their values in a 2D integer array.

**How to approach this task**

1. Declare an array with the required rows and columns.
2. Read the text from each JTextField.
3. Convert each text value into an integer.
4. Store the converted value at the correct array position.

5. Return the completed array.

Hint: Investigate the Java methods getText() and Integer.parseInt().

**Why?**

Text fields store user input as strings, but mathematical calculations require numeric values.

**Task 5.2: Implement the Calculate button**

When Calculate is clicked, the application must:

1. Capture the marks from the GUI.
2. Create an object of the StudentMarks class.
3. Pass the captured array into the constructor.
4. Call the calculation methods.
5. Display the results in the text area.
6. Update the Students Processed label.

**How to approach this task**

Use a button's ActionPerformed event in NetBeans.

Investigate how setText() can be used to display results.

**Why?**

This task connects the user interface to the business logic implemented in your class.

**Task 5.3: Validate user input**

Your application must reject invalid marks.

Test the following:

| Input | Expected behaviour |
|---|---|
| 75 | Accepted |
| 0 | Accepted |
| 100 | Accepted |

| | |
|---|---|
| 120 | Rejected |
| -5 | Rejected |
| abc | Rejected |
| Empty field | Rejected |

Hint: Research try, catch, NumberFormatException, and JOptionPane.

**Activity 6 – File handling**

**Task 6.1: Save student marks**

When the Save button is clicked, save the captured marks and calculated results to a file named studentmarks.txt.

Your file must follow this structure:

STUDENT MARKS REPORT

**********

Student 1: [Test 1], [Test 2]

Student 2: [Test 1], [Test 2]

Student 3: [Test 1], [Test 2]

Total Marks: [Calculated total]

Average Mark: [Calculated average]

Passing Marks: [Calculated count]

Failing Marks: [Calculated count]

Students Processed: [Calculated count]

**********

**How to approach this task**

Investigate these Java classes and methods:

| Java feature | Purpose |
|---|---|
| FileWriter | Write text into a file |
| write() | Write the report |
| close() | Close the file |
| IOException | Handle file errors |

**Why?**

File handling allows the application to preserve data after it has been closed.

**Task 6.2: Load student marks**

When Load is clicked, the application must:

1. Open studentmarks.txt.
2. Read the saved student marks.
3. Extract the two marks for each student.
4. Populate the six GUI text fields.
5. Recalculate and display the results.

**Hints**

Investigate FileReader, BufferedReader, readLine(), and split().

For example, consider how you could separate two values stored as 70, 60.

**Why?**

Loading data allows the user to continue working with previously saved information without re-entering all the marks.

Challenge: What should the application display if the user clicks Load before the file has been created?

**Activity 7 – Unit testing**

**Task 7.1: Create a test class**

Create a JUnit test class named StudentMarksTest.

Use the sample marks from Activity 1.

You must create these tests:

| Test method | Purpose |
|---|---|
| GetPassCount_ReturnsNumberOfPasses | Verify the number of marks at least 50 |
| GetFailCount_ReturnsNumberOfFails | Verify the number of marks below 50 |

**How to approach this task**

For each test:

1. Prepare the sample 2D array.
2. Create a StudentMarks object.
3. Determine the expected result manually.
4. Call the method being tested.
5. Compare the expected and actual results using assertEquals().

**Why?**

Unit testing verifies that the calculation logic produces the correct output independently of the GUI.

**Additional challenge**

Create two extra tests:

- Verify that a mark of exactly 50 counts as a pass.
- Verify that GetAverageMarks() returns the correct average.

**Activity 8 – Final practical testing**

Use this checklist before submitting your work.

- My JFrame opens correctly
- All six text fields capture marks
- The 2D array stores marks correctly
- My class implements every interface method
- The total and average calculations work
- Pass and fail counts work
- Invalid marks are rejected
- Calculate displays the correct report
- Save creates studentmarks.txt
- Load restores previously saved marks
- Clear resets the GUI
- Both required unit tests pass

**The most important concepts for your practical exam**

1. Interface: Defines the methods your class must implement.

2. 2D array: Stores multiple students' marks in rows and columns.

3. Nested loops: Process every mark to calculate totals and pass/fail counts.

4. GUI events: Execute code when a button or menu item is clicked.

5. File handling: Saves the marks to a text file and reads them back.

6. Unit testing: Confirms that calculation methods return the correct results.

The key workflow to remember is:

**Capture from JTextFields → Store in 2D array → Pass array to class → Calculate → Display → Save to file → Load from file.**